

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE



ROBOTIQUE, 2025-26

Master thesis

DASH-01 :
RL Framework to Develop a Bipedal
Humanoid Robot Aimed at Breaking the
Speed Record

REHAssist

13.04.2025 - 04.09.2025

Author:

Nathann MORAND
Robotics M.Sc.

Supervisors:

Dr. Mohamed BOURI

Abstract

DASH-01 is a 15 kg biped with six quasi-direct-drive actuators, a parallel 4-bar knee and a passive ankle. It was built at REHAssist to answer one question: how fast can a cheap custom biped run? This thesis, although RL oriented in the title, takes the platform from a partially finished mechanical design to a modelled machine that can be simulated and for which RL policy can be trained against. Finally, this thesis assesses the performance envelope of the platform and the policy.

The work covers the whole chain. A CAD-to-MJCF pipeline closes the kinematic loop that URDF cannot express. A measurement campaign to model the plant parameter. A first-principles feasibility sweep bounds the running speed the mechanism admits.

The curriculum has been shown to converge to a running policy within literature reported training steps.

On this platform the binding constraint is the plant rather than the algorithm. The thesis therefore ends in a ranked set of mechanical changes rather than in a controller.

Table of Contents

Abstract	2
1 Introduction and context	4
1.1 The platform and the objective	4
1.2 Change over the year about the objective	4
1.3 Contributions	4
2 Background and literature review	6
2.1 Definitions	6
2.2 Abbreviations	6
2.3 Bipedal locomotion approach	6
2.4 Reinforcement learning for locomotion	8
3 The robot and its digital twin	11
3.1 Dash-01	11
3.2 Selecting hardware	12
3.3 The digital twin	14
3.4 Digital twin validation and plant identification	14
3.5 Deployment stack	18
3.6 Running speed ceiling estimation	20
4 Reinforcement learning: attempts and architecture	23
4.1 Problem formulation and general RL architecture	23
4.2 The action-space ladder	23
4.3 Curriculum design	25
4.4 The pitch wall	25
5 Results	27
5.1 Hardware delivered	27
5.2 Running speed ceiling estimation	27
5.3 Reinforcement learning results	27
6 Discussion	28
6.1 Method-level lessons	28
6.2 Future work	28
7 Conclusion and future work	29
7.1 What was set out to do, and what was delivered	29
7.2 Limitations	29
7.3 Future work	29
8 Appendix	30
8.1 From CAD to a simulable model	30
8.2 Component databases	30
8.3 Logbook	31
References	33

1 Introduction and context

This chapter states the objective, the platform, and the scope. It closes with the contributions.

1.1 The platform and the objective

DASH-01 is a bipedal robot built at REHAssist. It carries two legs, six quasi-direct-drive actuators, a parallel 4-bar knee, a passive ankle spring and a single-board computer. The hardware budget was roughly CHF 5000. Figure 3.1 shows the machine as modelled.

The objective at kickoff was maximum running speed, with the bipedal 100m record as the ambition. That record have moved a few time in the last months and now stands at 8.86s, or about an average of 40.63 km/h.[2] Faster than Usan Bolt’s 9.58s. Before the begining of this thesis the record was held by Cassie in 2022 [5] at 24.73 s for the 100m dash.

The project inherited a partially finished robot. The work reported here therefore starts at modelling, and at the selection of the non-mechanical subsystems. It then runs through characterisation and the training of a policy.

1.2 Change over the year about the objective

The record moved faster than the thesis. New bipedal speed marks now appear at roughly a monthly cadence, from teams with institutional budgets.

The measured plant does not admit record pace. Section 3.6 bounds this mechanism at about 6 m/s. The bound comes from first principles, by using the workspace of the leg and the impulse it can deliver.

The objective of this thesis therefore restated as a measured demonstrator plus a reusable method. The deliverable is the chain from CAD to a trained policy.

The problem this addresses. Reinforcement learning has produced legged-locomotion results on research platforms. Reproducing them on a new, custom, imperfectly known robot is non trivial. The plant parameters are uncertain. The actuators hide undocumented internal control loops. The sensors are commodity parts. The gap between “reinforcement learning walks in simulation” and “reinforcement learning walks on this robot” is dominated by modelling fidelity, actuator bandwidth and calibration.

Sensing scope. Proprioception and one base inertial measurement unit are the only sensors. Foot impact sensors were dropped due to lack of time rather than on evidence. The hardware provision for them remains in place, as four analogue inputs on the sensor hat. The literature prior that foot feedback helps convergence is therefore untested here.

1.3 Contributions

- **A CAD-to-MJCF pipeline**, with the loop-closure treatment URDF cannot express. The model was then validated against the backdriven workspace (Section 3.3).
- **A measured plant** (Section 3.4). It covers weighed segment masses, a torque constant identified by a known-mass method that breaks the degeneracy between K_t and gravity, the Bode response of the drive’s internal position loop, inertial-sensor noise characteristics, loop timing and the CAN protocol.
- **A first-principles capability analysis** of running speed on that plant, validated against MuJoCo. A mass-distribution sensitivity analysis is also performed to rank the design levers

(Section 3.6).

- **An action-space and curriculum design** for the learned controller. Multiple action spaces were tested, and a curriculum was built to converge to a running policy on the measured plant (Sections 4.2.4 and 4.3).
- **A deployment stack on the robot.** It is a 200 Hz daemon on stock Linux, a web bring-up instrument that was used to perform the measurement of the robot characteristics. The deployment stack also includes safety mechanisms to prevent the robot from exiting the safe operating envelope (Section 3.5).
- **Two component databases**, covering 140 commercial actuators and roughly 430 battery cells. They were used to validate the validity of the choice of the robot part. (Section 3.2, Section 8.2).

2 Background and literature review

This chapter provides a brief context for bipedal locomotion and, more specifically, for the Reinforcement Learning framework used in the literature to address this problem.

2.1 Definitions

First, a few definitions are provided, as the following terms will recur throughout this thesis.

Underactuation is the presence of more degrees of freedom than independently actuated degrees of freedom.

Unilateral contact means that a foot can push against the ground but cannot pull it, and that the resulting contact force must lie within the **friction cone**.

The point at which the resultant contact force acts on the ground is the **centre of pressure**, which is confined to the contact patch.

Duty factor is the fraction of a gait cycle during which a given foot remains in contact with the ground. A **flight phase** is an interval during which neither foot is in contact with the ground.

The **sim-to-real gap** is the performance mismatch observed when transferring a policy from a simulated plant to the physical robot. This mismatch generally arises because the real plant lies outside the distribution of plants and disturbances encountered during training.

In the context of this thesis, **failure** refers to the termination of an episode following body contact with the ground or leaving the measured workspace.

Stability margin, in the context of Reinforcement Learning, is not necessarily expressed as a classical analytical stability margin. In this thesis, robustness is instead assessed through quantities such as the number of surviving episodes and the variance across random seeds.

Plant, borrowed from control theory, refers to the physical system to be controlled, in this case the robot.

2.2 Abbreviations

RL : Reinforcement Learning

CoM : Centre of Mass

ZMP : Zero Moment Point

2.3 Bipedal locomotion approach

Bipedal locomotion can be approached in different ways, depending in particular on the speed of motion and on the assumptions made about contact and dynamics.

The simplest case is a robot with flat feet walking on flat terrain at sufficiently low speed that inertial effects can be neglected. This was the approach used for WABOT-1 in 1973 [25].

In this case, maintaining stability can be reduced to keeping the projection of the CoM inside the support polygon formed by the feet.

As the robot moves faster, inertial effects can no longer be neglected. The relevant quantity is then not simply the projection of the CoM, but the Zero Moment Point (ZMP), which must remain within the support polygon. This approach was used, for example, in ASIMO [26].

Running introduces a flight phase, which is incompatible with the previous approaches that require at least one foot to remain in contact with the ground at all times.

To enable running, Raibert [19] proposed a foot-placement method based on the following relation:

$$x_f = \frac{1}{2} \cdot \dot{x} \cdot T_s + k \cdot (\dot{x} - \dot{x}_d) \quad (2.1)$$

where x_f is the fore-aft foot placement, measured from the hip at touchdown, in metres; $\dot{x}$ is the current forward speed in metres per second; $\dot{x}_d$ is the commanded forward speed, in metres per second; T_s is the stance duration, in seconds; and k is a velocity feedback gain, also expressed in seconds.

The first term corresponds to the neutral point, which produces no net fore-aft impulse. The second term provides velocity feedback: when the robot is moving faster than commanded, the foot is placed further forward, producing a decelerating effect during stance.

This lineage runs through systems such as BigDog [18] and later Atlas.

The underlying idea is that the gait can be stabilised, to a significant extent, by placing the feet at the appropriate location and time.

This also makes it possible to build running robots with point-contact feet, rather than requiring the large flat feet used by earlier approaches.

The approach was subsequently refined by solving the foot-placement and whole-body motion problem online, leading to what is commonly formulated as a Model Predictive Control approach [11].

This approach was central to the development of the early dynamic legged robots developed by Boston Dynamics.

These approaches rely on a sufficiently accurate model of the plant.

In practice, however, the plant parameters are only partially known and may vary with configuration, load, contact conditions, and hardware.

Classical model-based controllers are therefore designed around a particular model and possess a finite robustness margin with respect to modelling errors and disturbances. Beyond this margin, the controller may no longer be able to stabilise the system.

Reinforcement Learning provides an alternative way of obtaining this robustness. Rather than explicitly deriving a controller from a single plant model, RL trains a policy over a distribution of simulated plants. The policy can therefore be exposed to variations in plant parameters during training, for example through domain randomisation, with the objective of improving its robustness to model mismatch.

The problem is further complicated by the unilateral and saturating nature of ground contact. A foot can only push against the ground, not pull it, and the tangential component of the contact force is bounded by the friction cone. Exceeding this bound results in slipping.

A second complication is the timescale of the dynamics. A legged robot can, to a first approximation, be considered as an inverted pendulum with characteristic length h .

In the absence of corrective input, the characteristic timescale of the inverted pendulum is on the order of

$$\tau \sim \sqrt{\frac{h}{g}}, \quad (2.2)$$

which corresponds to approximately 310 ms for a characteristic length of 1 m.

During running, however, contact with the ground may last only a few tens of milliseconds. Within this timescale, the robot must measure the state of the system, estimate the relevant dynamics, compute a corrective command, communicate it to the actuator, and physically produce the required motion.

This places significant requirements on sensing, computation, communication latency, and actuator bandwidth.

The robot considered in this thesis has a measured actuator bandwidth corresponding to a 2 Hz pole in servo mode and approximately 20 Hz in MIT mode.

This limitation will become relevant again when discussing the control architecture and the sim-to-real problem later in this thesis.

2.4 Reinforcement learning for locomotion

2.4.1 Actuator control architecture

Several actuator-level control architectures have been used in the literature. The main ones considered here are direct torque control, direct position control, and impedance control.

Direct torque control allows the policy to command the actuator torque directly, giving it the highest level of control authority.

Direct position control allows the policy to command a position target, which is then tracked by a low-level controller.

Impedance control allows the policy to command a desired mechanical behaviour through parameters such as position, stiffness, damping, and, depending on the implementation, torque-related terms.

The latter provides a compromise between control authority and the requirements imposed on the hardware in terms of latency and bandwidth.

Direct torque control provides the policy with the highest level of flexibility, but also imposes demanding communication and control-loop timing requirements.

In simulation, the policy can directly command joint torques at a high frequency [7].

On the physical robot, however, closing such a fast loop requires the policy, communication bus, actuator controller, and state estimator to operate with sufficiently low latency.

In the present hardware architecture, this approach was not practical through the CAN communication bus at the required frequency.

Direct position control substantially reduces these requirements by delegating the low-level actuator dynamics to the motor controller.

Position targets passed to the joint controller are used by several learned locomotion approaches [24, 20].

Two variants were investigated in this thesis: conventional smart-servo position control and the MIT control mode with fixed compliance parameters [10].

Both allow the RL policy to operate at a lower frequency while the actuator controller handles the faster inner control loop.

Impedance control provides an intermediate level of control authority [4].

Instead of directly specifying actuator torque, or being restricted to position targets, the policy can influence the desired mechanical behaviour through position, stiffness, damping, and/or torque-related parameters.

This avoids the need to close the torque loop at high speed over the communication bus while providing the policy with greater freedom than pure position control.

It also exposes more of the actuator dynamics to the policy and can reduce the sim-to-real gap compared with a smart-servo interface when the frequency and parameters of the inner control loop are not fully exposed.

2.4.2 Gait prior

For the purpose of comparing these approaches, the literature can be organised along a second dimension: the amount and location of prior structure imposed on the gait.

A strong prior can be introduced through a **reference trajectory**.

In this case, the policy is primarily trained to track an authored motion. This approach was, for example, explored on the Cassie robot [27].

This strongly restricts the reachable set of motions and can substantially simplify learning.

However, the quality of the resulting behaviour is bounded by the choice of reference trajectory: there is no guarantee that the selected trajectory is optimal for the particular robot morphology and dynamics.

A less restrictive prior can instead be introduced through a **trajectory generator** whose parameters are modulated by the policy [9].

This preserves the assumption that locomotion should have a structured, periodic form while allowing the learned controller to search for a gait adapted to the plant.

Trajectory generators can themselves provide different levels of freedom over the gait parameters.

At the stricter end of this generator-based formulation, the policy only modifies a small number of **high-level gait parameters**, such as phase, frequency, stride length, or amplitude [3].

This is similar to the reference-trajectory approach in the sense that the policy operates within a predefined family of trajectories. However, instead of learning the low-level joint commands required to reproduce a reference gait, the policy learns the high-level parameters that generate it.

This provides a relatively small search space but limits the set of gaits that can be represented.

A more moderate prior allows the policy to modify the **complete trajectory**. Rather than predicting every trajectory sample independently, the possible trajectories can be represented using a low-dimensional basis, such as Fourier series [22] or Bézier curves.

The policy then modifies the coefficients of this representation.

This retains useful structural assumptions such as periodicity and smoothness, while allowing the shape of the gait itself to become a learned variable.

It is also possible to use **neither a trajectory generator nor an explicit gait-shape prior**.

In this case, the policy learns the complete joint trajectory directly [6].

Periodic locomotion can still be encouraged through reward shaping, but periodicity then becomes an emergent property rather than an explicit constraint.

Such an approach provides a larger reachable set, but considerably increases the exploration and learning problem. There is also no guarantee that the learned solution will converge to a periodic gait.

Finally, the prior, when present, can be introduced at different levels: through the **reward** [23], through a **reference motion** [16], or through an authored **trajectory generator** [9, 3].

These choices determine how much of the gait is fixed and how much remains available to the policy at run time, at the expense of increasing or reducing the complexity of the learning problem.

2.4.3 Curriculum design

The third major design dimension is the curriculum used to train the policy.

To help the policy converge, the training environment can be modified to make the task easier at the beginning and gradually increase its difficulty as the policy improves.

The literature has explored a variety of curriculum-design choices, including the order in which different tasks are presented, the rate at which difficulty is increased, and the specific parameters that are modified to create the curriculum.

Terrain and command curricula. A common scheme starts every agent on flat ground with a narrow command range.

As its performance exceeds a predefined threshold, the agent is promoted to rougher terrain or a wider velocity range, and can be demoted otherwise [20].

Difficulty therefore tracks each agent’s competence rather than following a fixed schedule.

This approach is well suited to massively parallel training, where each agent can be promoted or demoted independently.

Automatic domain randomisation. Rather than fixing the domain-randomisation range before training, the range can be widened automatically once the policy exceeds a performance threshold at its current extremes [15].

The randomisation range itself becomes the curriculum parameter, expanding only as fast as the policy can tolerate.

This approach requires a reliable in-training performance signal.

A noisy signal can cause the range to grow faster than the policy can adapt, destabilising training.

Whether the curriculum targets terrain, command range, or randomisation range, an adaptive schedule removes the need to manually tune a progression rate.

In its place, the design requires a performance threshold and a metric with which to evaluate it.

This introduces a problem similar to that encountered in reward design: if the metric does not accurately represent the intended notion of competence, the agent may optimise the metric rather than the desired behaviour.

3 The robot and its digital twin

This thesis needed to first finish the Dash-01 robot. This implied to choose the computer, its actuator and the communication bus. It also needed a battery.

Once that was done, its digital twin was develop from the CAD data and validated against measured value from the assembled robot. Finally the model what could not be derived from the cad was measured on the real robot (like actuator bandwidth) to limit the sim2real risk.

3.1 Dash-01

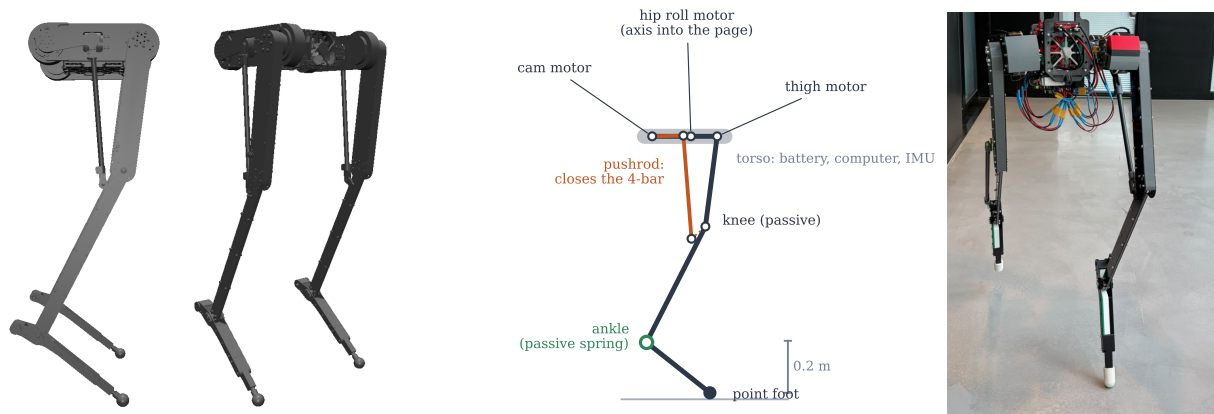


Figure 3.1: DASH-01, Simulated model and real-life picture.

	Hip roll	Cam, thigh	Segment	Mass	Count
Part	AK60-39	AKE90-8	Base	5764 g	1
Reduction	39:1	8:1	Hip, with two motors	3271 g	2
Peak torque, datasheet	72 N·m	170 N·m	Cam	66 g	2
Peak torque, measured	61.2 N·m	144.5 N·m	Thigh	483 g	2
Continuous torque	24 N·m	55 N·m	Push rod	71 g	2
No-load speed	10.3 rad/s	22.0 rad/s	Ankle spring assembly	249 g	2
K_t (line-to-line)	0.12	0.272 N·m/A	Shin	324 g	2
R (line-to-line)	0.60	0.164 Ω	Foot	222 g	2
$K_m = K_t/\sqrt{R}$	0.15	0.674	Total	15 136 g	
Reflected armature	0.046	0.0216 kg·m ²			
Mass, unit price	0.75 kg, 393	1.40 kg, EUR 424			

Table 3.1: The six CubeMars quasi-direct-drive actuators (left) and the weighed segment masses (right). Measured peak torque is the datasheet figure scaled by the effective joint K_t of Section 3.4.2.

Each leg carries three motors. Hip roll sits at the base, and a cam and a thigh axis drive the sagittal plane. The knee is passive and follows through the 4-bar. The ankle is passive and follows through a spring (not rendered). During the course of the project, the spring was proved to be too weak to support the robot's weight, and the ankle was therefore locked in a neutral position with a carbon tube. The mass in the simulation where updated to reflect that change.

3.2 Selecting hardware

3.2.1 Actuators choice

A database to select the actuator was assembled. Major robotic actuator manufacturers website was scraped for datasheet and a large language model was used to parse each datasheet. The resulting database holds 140 entries across 11 suppliers, listed in Table 3.2.

Manufacturer	Actuator count
ZeroErr	32
MyActuator	29
CubeMars	22
SteadyWin	21
Maxon	12
MAB Robotics	8
RobStride	7
HEBI Robotics	6
Soceboz	1
RSL-ETH	1
mjbots	1
Total	140

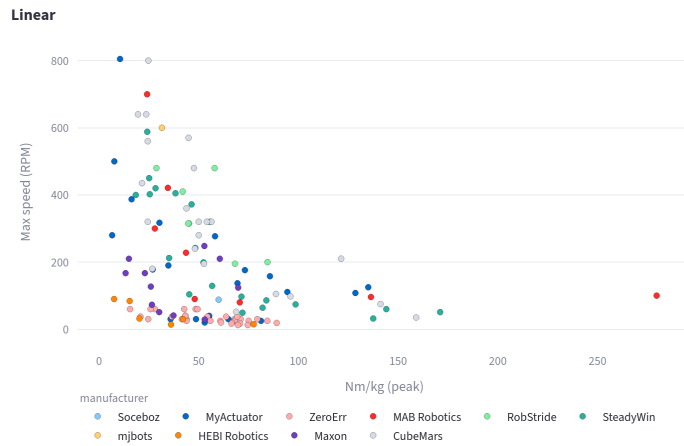


Table 3.2: Actuator manufacturers represented in the database

Figure 3.2: Excerpt of the dashboard, showing the actuator database used to select the drive.

The database was compiled into an interactive dashboard, *motormap* [13], built from the `actuatorReview` code [12]. The choice of the actuator itself is detailed in the master’s thesis of M. Aebi [1]. The final motor choice was the CubeMars AK60-39 for the hip roll and the AKE90-8 for the cam and thigh. The AKE90-8 are rated for up to 72A peak while the AK60-39 is rated for 17 A peak.

3.2.2 Battery choice

A database of roughly 430 commercial cells was built based on the same principle being the actuator database. Knowing that a 48V pack is required, the database was filtered to cells that can be used in a 13S configuration. It was also known from the motor section that the pack needed to deliver at least 160 A peak with the assumption that only half of the motor would be at maximum torque at once. From them, a 13S2P pack of EVE INR21700-50PL was selected. The selection criterion was a good compromise between energy density, power density and cost with the hard constraint of having a 48V pack (highest voltage before requiring different motor drivers).

The pack can be monitored over bluetooth and its limit configured there using the DALY bms app. A can interface is also available but not used in this project due to lack of time.

The internal resistance is stated by the manufacturer to be less than 10 mOhm per cell. When drawing the full 360 A peak, the voltage drop is therefore expected to be up to 23 V which is quite significant. In practice, the battery will realistically reach peak of 150 A but not much more. This means a drop of 10V which should not trip the BMS low-voltage protection given the battery is fully charged. Due to the lack of voltage feedback on the motor drivers, this figure has not been verified on the real robot.

Parameter	Specification
Cell	EVE INR21700-50PL, 5.0 Ah
Configuration	13S2P
Nominal voltage, capacity	48.1 V, 10 Ah (468 Wh)
Continuous rating	100 A, cell-limited
Peak current	360 A for 2 s
Mass (BMS, casing)	3.5 kg
Power density (peak)	4900 W/kg
Power density (cont.)	1370 W/kg
Energy density	133.7 Wh/kg

Table 3.3: Specification of the selected 13S2P battery pack.

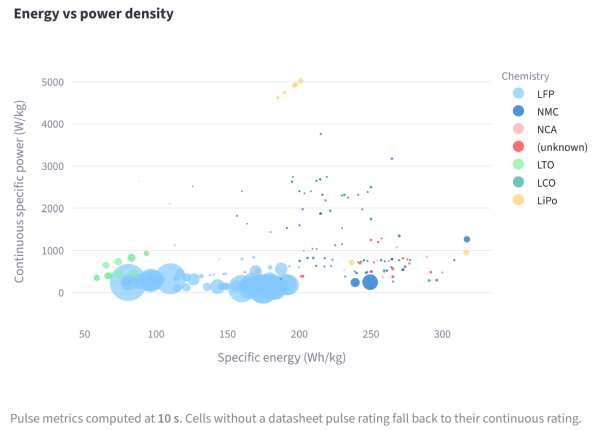


Figure 3.3: Excerpt of the dashboard, showing the cell database used to select the pack.

3.2.3 Compute, CAN bus and sensing

The following Table 3.4 summarises the choice of the compute, sensing and network bus.

Compute	
Board	Raspberry Pi 3B, stock non-real-time Linux
Control loop	200 Hz
Sensing	
IMU	ICM-20948, 10-axis, on a Waveshare Sense HAT (B)
IMU interface	I2C, ± 4 g, ± 1000 dps, 200 Hz sampling
Analog input	4-channel ADC on the same HAT
Network bus	
Controller	mcp251xfd over SPI, dual-channel CAN HAT
Topology	two independent 1 Mbit buses, one per leg
Measured load	26 % of each of the two buses (polled at 200 Hz)

Table 3.4: The robot’s non-mechanical architecture: compute, sensing and the CAN network bus.

The board is a **Raspberry Pi 3 Model B**, running stock non-realtime Linux. It was chosen over the laboratory’s usual BeagleBone Black because the author could not handle the slow SSH session. Furthermore, the multicore nature of the Pi allows for better performance in handling the control loop and data processing tasks. An `mcp251xfd` controller over SPI provides CAN as two independent 1 Mbit buses, one per leg. The Pi also hosts a WiFi access point (SSID : "runningmachine", pwd:"runningmachine") and the web interface ("http://192.168.4.1:8080/"), through which most measurements in Section 3.4 were taken.

Section 3.4.4 present the measurement of the timing of the control loop and the CAN bus.

Aside the motor encoders and current feedback, the only proprioceptive sensors is the IMU located on the base of the robot. Contact sensors were considered and never fitted due to lack of time. The IMU unit is an **ICM-20948** on a Waveshare Sense HAT (B), read over I2C.

3.3 The digital twin

A simulation ready model was derived from the CAD data. The model is exported in two formats, URDF and MJCF. The detail procedure to export the model is described in the appendix Section 8.1. This took a lot of time and effort due to lack of experience and the poor documentation of the plugin used to export the model. The final recipes consisted into exporting each segment of the robot as a STEP file with the origin at the joint (except for the base which was in its center). Care was taken to ensure the references axis were aligned in a consistent manner across the segments. The robot model was reassembled in autodesk Fusion 360 and exported to URDF using the ACDC4Robot plugin[17]. The kinematic loop where broken in the URDF and closed in the MJCF using an equality constraint between two sites, one on the pushrod and one on the shin. This was necessary because URDF is a tree format and cannot express kinematic loops.

3.4 Digital twin validation and plant identification

The digital twin is validated against the real robot for the kinematic and dynamic properties. The model is also refined with measured parameters that cant be accurately estimated from the CAD model, to reduce the sim2real gap.

3.4.1 Joint Limits validation

The cam and thigh joints close a passive 4-bar loop through the pushrod and knee. Their safe ranges do not compose into two independent intervals: a per-joint minimum and maximum would allow poses the mechanism cannot assemble. I derive the safe envelope directly in actuator space instead. I hold each leg's three drives current-free and backdrive it by hand through its full travel, logging the raw actuator angles [14].

Figure 3.4 shows the result for both legs: a plain interval for hip roll, and a two-dimensional band for the coupled (cam, thigh) pair. The demonstrated hip roll range spans 42.8 deg to 126.1 deg on the left leg and 66.5 deg to 147.7 deg on the right; I erode both by 3 deg for a safety margin and store the eroded interval as the hip roll limit. For the knee pair I rasterise the recorded (cam, thigh) samples onto a 1 deg grid, dilate it to close small sampling gaps, then erode it by the same 3 deg margin. The stored safe region covers 11.0 % of its own bounding box on the left leg and 9.4 % on the right, confirming the band is far from rectangular. The onboard safety layer checks every commanded pose against this grid before it reaches the drives (Section 3.5.2).



Figure 3.4: Joint-limit validation in actuator space: the coupled (cam, thigh) 4-bar band, back-driven by hand for each leg. The green region is the safe envelope enforced by the onboard safety layer.

3.4.2 Inertia matrix validation

Each actuated joint's torque constant K_t , effective inertia and friction (viscous and Coulomb) from two excitation profiles logged through the web interface [14]. A quasi-static sweep at 0.03 Hz holds the joint near equilibrium, so gravity dominates the measured torque. A single per-joint least-squares fit on the pooled data recovers K_t , the effective inertia and the friction terms together. The fit uses the model's gravity torque, computed through the closed 4-bar loop (Section 3.3).

A raw torque residual is hard to judge on its own, so it was converted into a mass error. It is an upper bound, since a part of the residual is due to friction and not the gravity torque. As it can be seen in figure Figure 3.5, the error is big but it allows to ensure that the mass model is in the right order of magnitude and that the gravity trend is correct. Since the mass error is bounded, it can be used for domain randomisation in the RL training to make the policy more robust to this uncertainty.

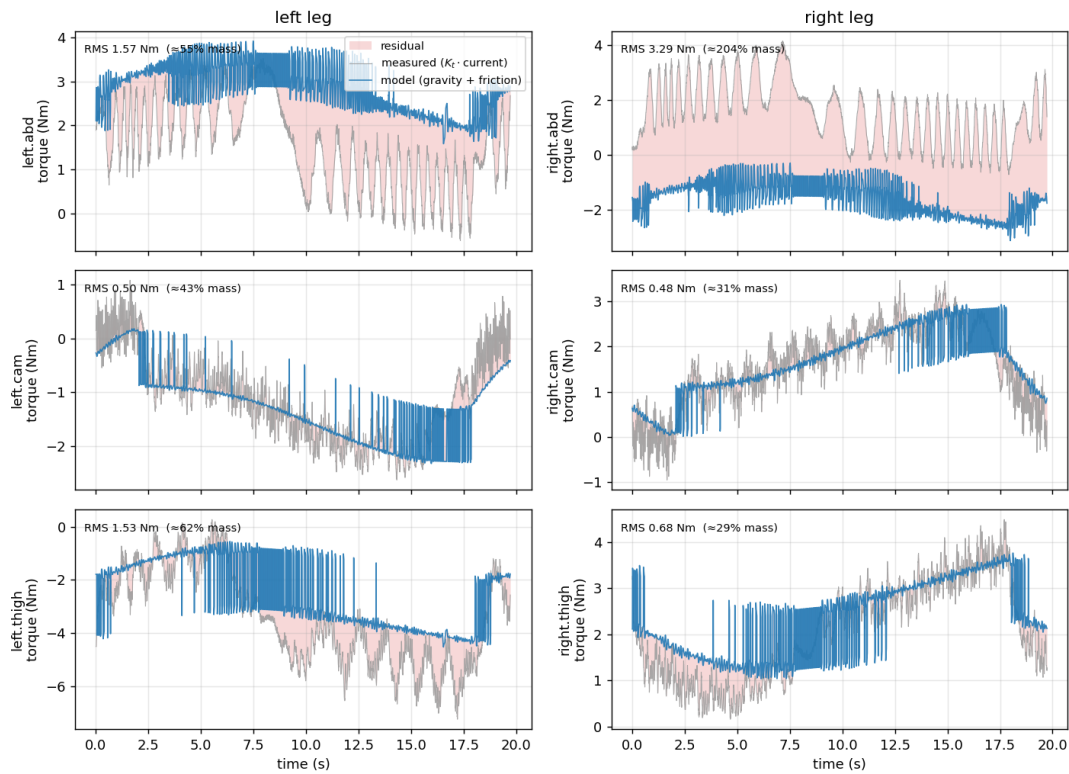


Figure 3.5: Gravity-model validation on the slow (0.03 Hz) quasi-static sweep: measured torque (K_t times current) against the model's predicted torque (gravity plus friction), residual shaded, per joint. The mass-equivalent error is the RMS residual divided by the RMS gravity torque. The fast ripple is likely the P-only position loop's stick-slip response due to friction.

3.4.3 IMU characterisation

at $\pm 4g$ and ± 1000 dps, with a 50 Hz internal low-pass and 200 Hz sampling. Measured at rest it gives 1.8 mg and 0.08 dps RMS noise. The noise density is $218 \mu g/\sqrt{\text{Hz}}$ and $0.0096 \text{ dps}/\sqrt{\text{Hz}}$. The gyro turn-on bias is repeatable at $[1.03, 0.11, -0.53]$ dps, and the in-run bias stability is 0.010–0.012 dps.

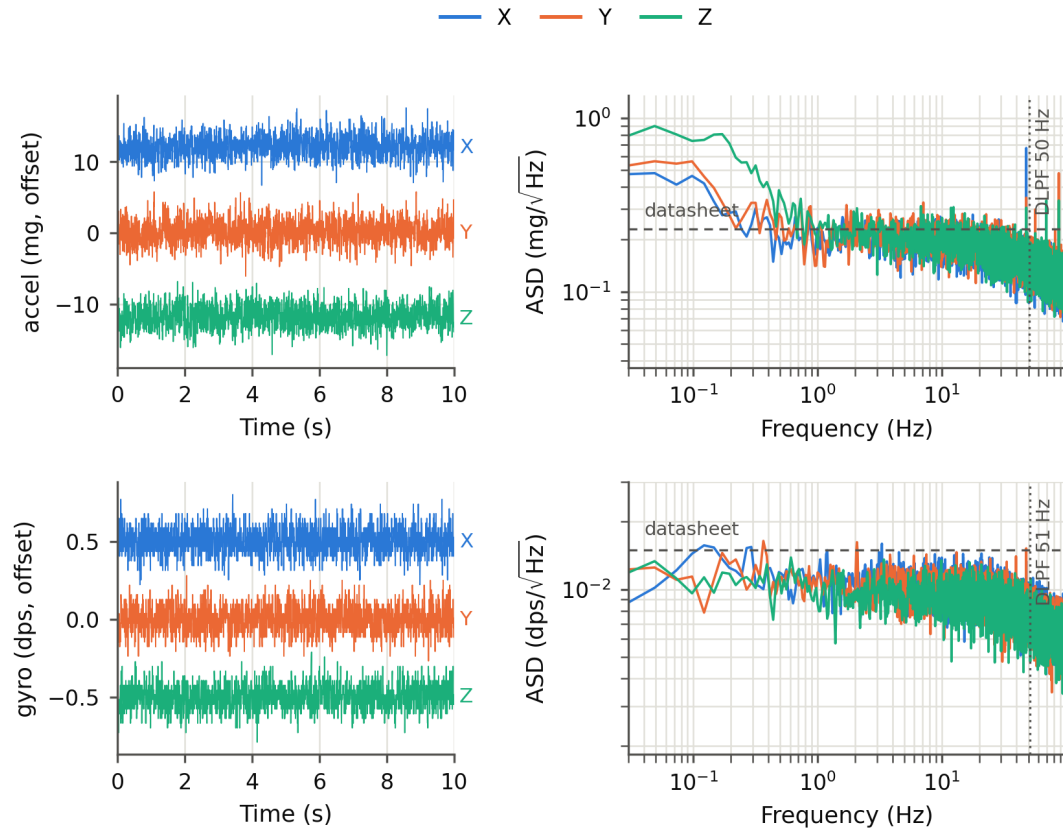


Figure 3.6: Noise characterisation of the base IMU (ICM-20948) sampled over 300s at

3.4.4 CAN bus latency and load characterisation

The whole control loop was benchmarked on the robot with the web interface running. These results were reused in the RL curriculum to make the policy robust to the timing jitter.

Rate	Mean period	Jitter (sd)	p99	Max	Budget used
50 Hz	20.148 ms	2.151 ms	22.41 ms	49.29 ms	23 %
100 Hz	10.000 ms	0.251 ms	10.40 ms	12.28 ms	37 %
200 Hz	5.000 ms	0.141 ms	5.21 ms	7.46 ms	50 %
400 Hz	2.504 ms	0.193 ms	3.24 ms	5.13 ms	73 %

Table 3.5: Measured closed-loop timing on the robot’s Raspberry Pi 3B, with the web interface competing for CPU. One tick drains both CAN buses, runs the inference of the policy and writes all the motors commands.

3.4.5 Actuator thermal characterisation

Not measured due to lack of time. The thermal lumped model parameters were estimated roughly from the mass of the actuator but not used in the simulation.

3.4.6 Battery characterisation

Not done due to lack of time, estimated from the data sheet internal resistance of the cell. This is further complicated by the lack of voltage feedback on the motor drivers, which prevents

measuring the voltage drop under load. In practice, this is not expected to be a problem because the driver are not used in torque control and thus it can excepted from the driver to be able to regulate it without taking the RL policy out of it's distribution.

3.4.7 Actuator bandwidth characterisation

The AKE90-8's closed-loop bandwidth was measured in both Servo and MIT mode.

Figure 3.7 shows the results. Servo mode reaches -3 dB at 1.95 Hz. MIT mode reaches -3 dB between 6.0 Hz and 20 Hz depending on k_p and k_d .

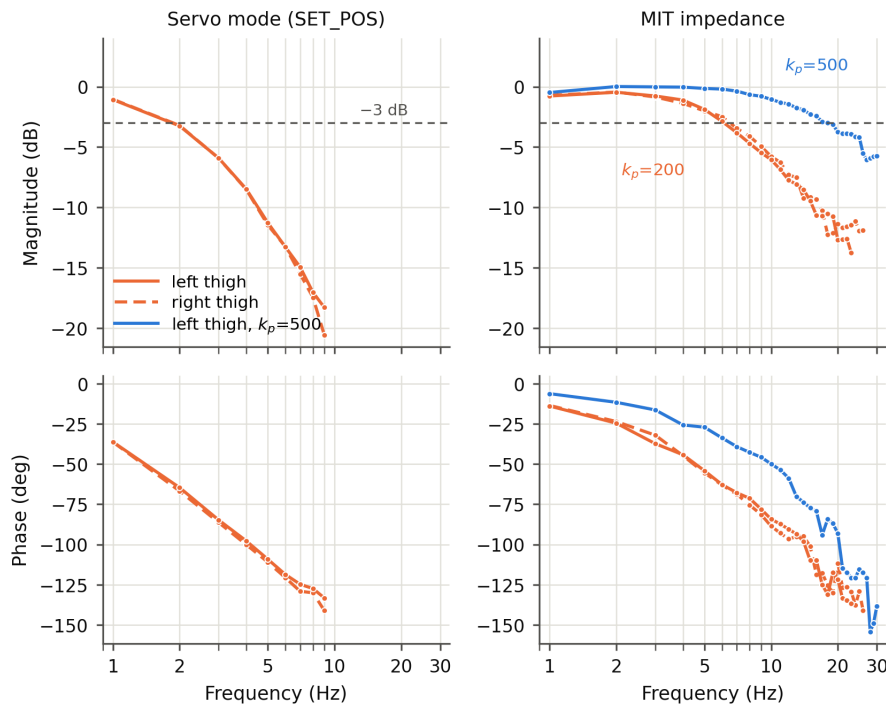


Figure 3.7: Measured closed-loop tracking a sine chirp and its response as the frequency increases in term of tracking error of the AKE90-8 drive: servo mode's position loop (free rotor) against MIT mode's position loop (free rotor).

3.5 Deployment stack

The policies of Section 4 have to run somewhere. To make streamline the process and simplify the measurement of the plant parameters, a web interface was built on the robot. A systemd entry as added at boot to start the web interface. The raspberry pi was configured to act as a wifi access point (SSID : "runningmachine", pwd:"runningmachine") Once connected to the network, the web interface can be accessed at ("http://192.168.4.1:8080/")

3.5.1 Calibration sequence

The actuator dont hold their position when powered off, so a calibration sequence is needed to zero the joints. The user need to hold the robot in a known configuration and run the calibration sequence which register the current joint position as zero. The user then need to move each joint in a specific direction and record that the angle in increase (otherwise, the joint direction need to be inverted by clicking a button in the web interface). Once the robot is zeroed, the workspace can be registered by backdriving the robot and recording the joint position at the limit of the

workspace. The web UI then clean the measurement to show the boudany and the user can draw / fill the safe workspace in the web interface. This is used to limit the joint position in the safety layer.

the IMU is also calibrated in the web interface by holding the robot still and recording the bias of the accelerometer and gyroscope. To ensure the axis are aligned, the user need to move the robot around an given axis of the base and check that the correct axis is increasing.

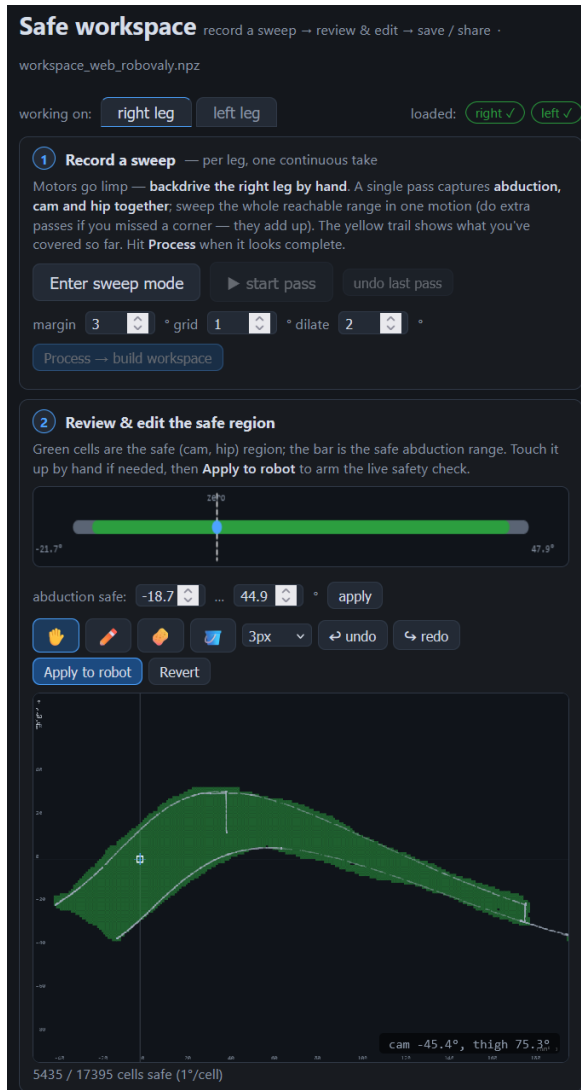


Figure 3.8: Workspace widget of the web interface, showing the backdriven joint envelope and the user-drawn safe workspace boundary.

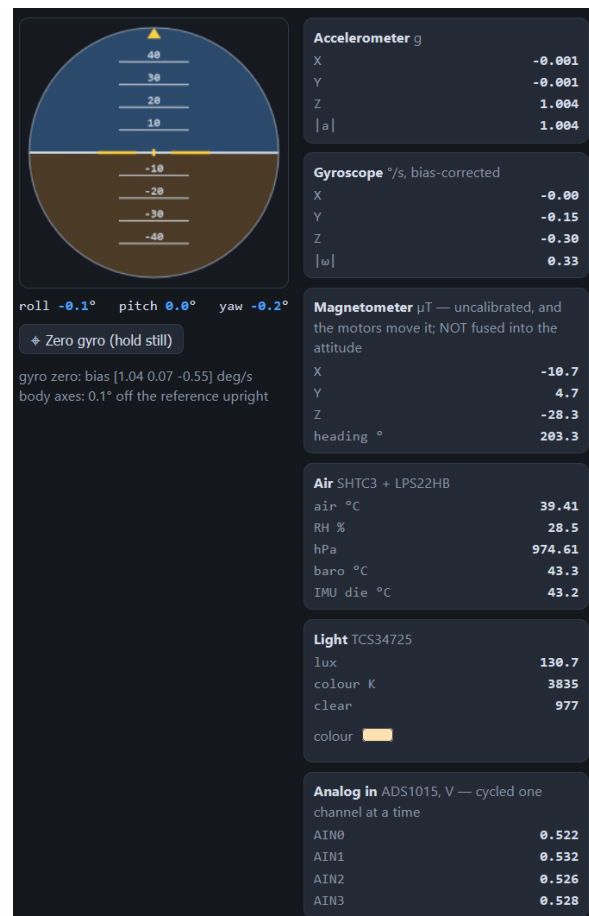


Figure 3.9: IMU widget of the web interface, used to record accelerometer and gyroscope bias and check axis alignment.

3.5.2 The safety layer

A safety layer was implemented to prevent the robot from exiting the safe operating envelope. It limits the joint position, the joint speed, the joint torque. If a tracking error greater than a threshold is detected, the robot is stopped. The user can also at any time stop the robot by clicking the E-STOP button in the web interface. This safety layer was thoroughly tested in simulation and on the real robot by trying to make the robot exit the safe operating envelope and was proven useful in a few occasion to prevent user error. The safety layer also implement a

black box recorder that record the joint position, speed, torque and the IMU data at 200 Hz if a new failure mode is triggered, it will be possible to investigate it.

3.5.3 Running widget

From the web interface it is possible to both record a gain by backdriving the foot of the robot and then to play it back at a wanted frequency. This is also where a policy can be selected and run on the robot.

3.5.4 Measurement widget

explain each widget and how it was used to measure the plant parameters.

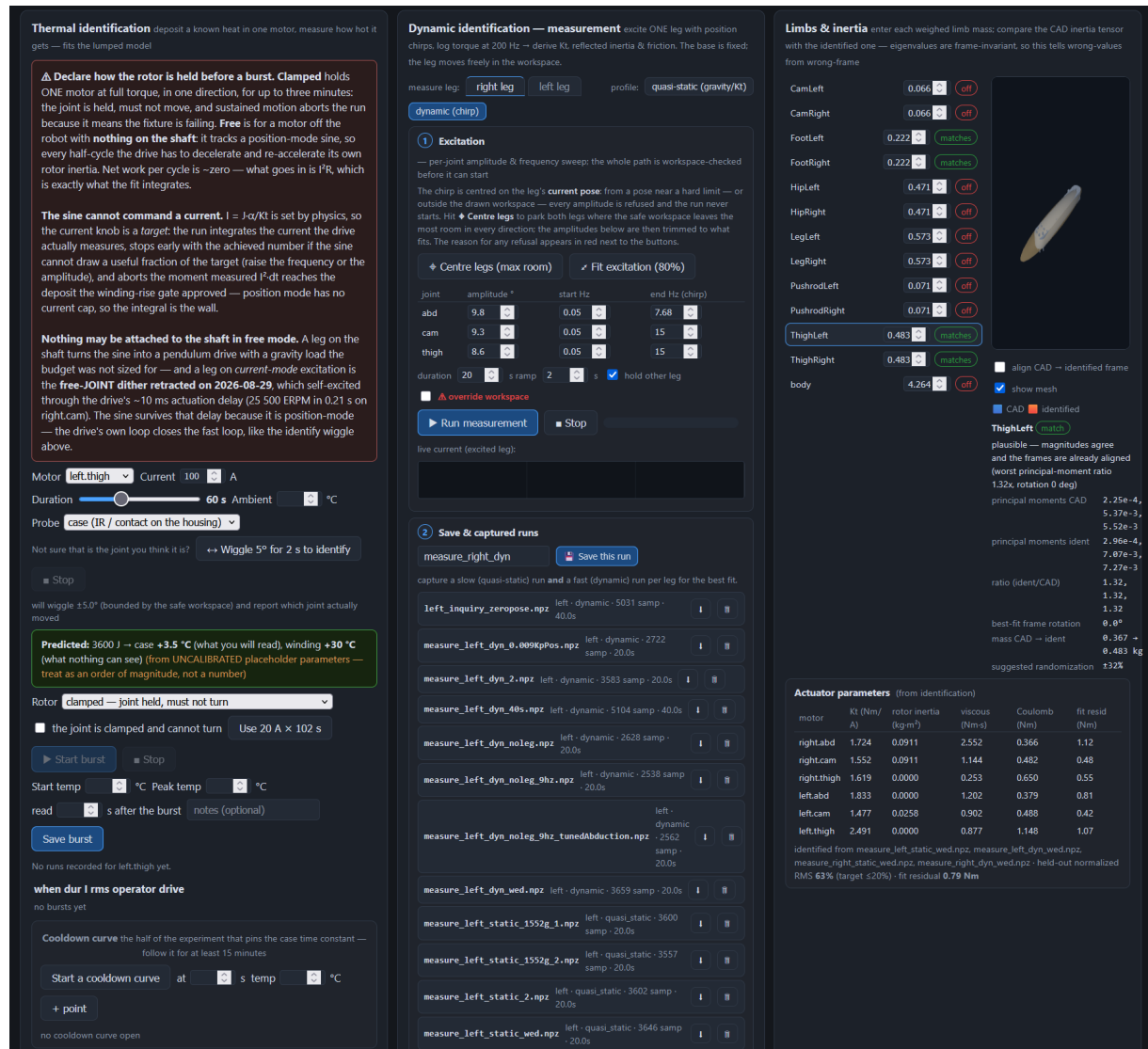


Figure 3.10: The three widget used to validate the robot parameters and limit the sim2real gap.

3.6 Running speed ceiling estimation

The physical speed ceiling was established independently of reinforcement learning by applying progressively stricter physical constraints to the same robot model. First, the body was fixed and

ground interaction removed with the leg free to move. This was used to determine the maximum feasible fore-aft foot sweep over the empirically verified workspace. The resulting speed was evaluated first from joint-speed limits alone, then with the measured torque-speed envelope and leg inertia, and finally with the measured actuator command latency. This progressively reduced the theoretical ceiling from 14.5 m/s to 7.4 m/s and ultimately 6.55 m/s, showing that actuator torque and leg inertia are the dominant limitations. The result is an optimistic upperbound because the leg carries no body load and balance and contact constraints are ignored.

A second analysis extends the bound to gaits involving a flight phase, where the rail argument no longer applies because ballistic flight can increase stride length. The maximum speed was instead constrained using an impulse balance: the available vertical foot force, obtained from the leg's force capability, was combined with the minimum time required to execute a stroke. As speed increases, stance duration decreases while the leg's minimum cycling period remains finite, eventually leaving insufficient stance impulse to support the robot's weight. This gives a slightly lower upper bound of approximately 6.2 m/s.

Model	Bound	Cost	Binding mechanism
Kinematic (no-load speed only)	14.5 m/s	–	joint speed through path Jacobian
+ torque-speed envelope, inertia	7.4 m/s	–49 %	stroke reversals, torque at speed
+ 7 ms MIT-mode latency	6.55 m/s	–12 %	delay caps usable stiffness
Flight-phase closure (any gait)	6.2 m/s	–	vertical-impulse starvation

Table 3.6: The speed-ceiling with refined modeling incorporating more realistic physical constraints.

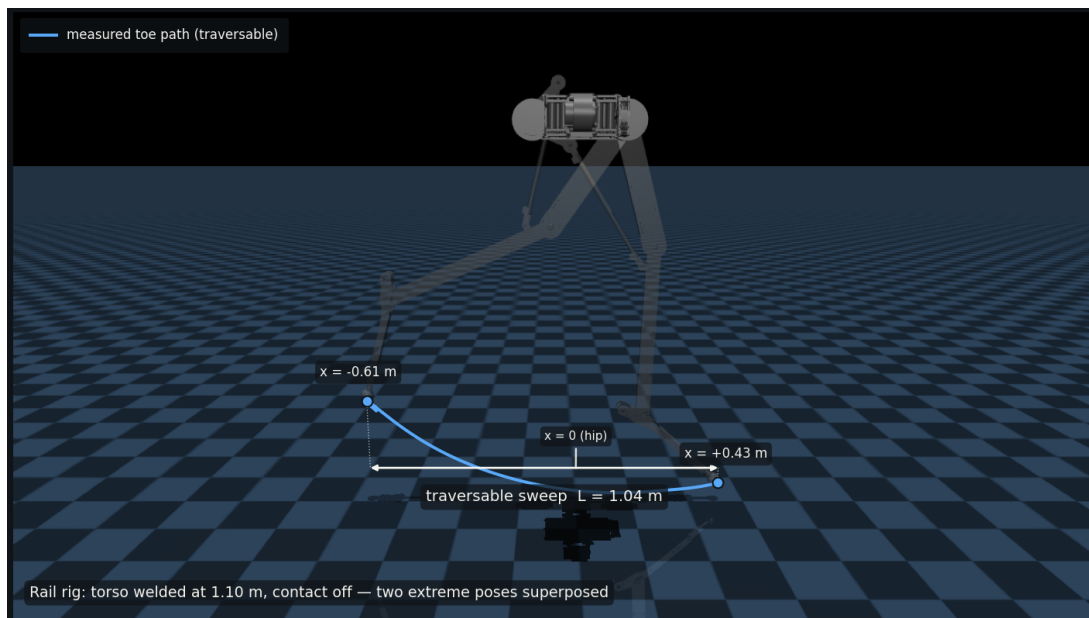


Figure 3.11: Workspace of the leg used to infer an optimistic upper bound on the running speed.

Static end-effector force capability over the recorded safe workspace (values capped at 3.5 BW; colour scaled to each map's 10th--90th percentile)

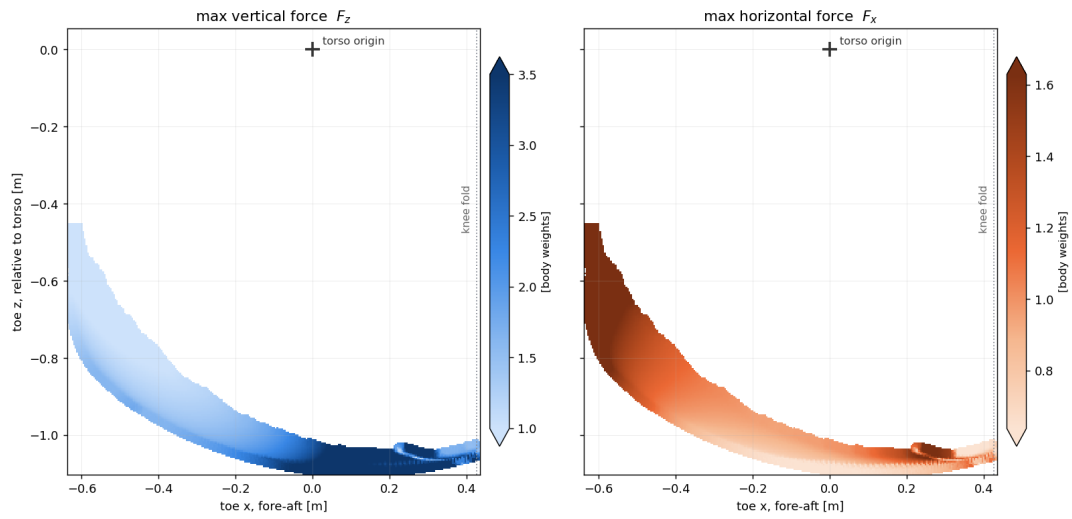


Figure 3.12: Map of the leg's maximum vertical force capability over its workspace, used to infer a more realistic upper bound on the running speed.

4 Reinforcement learning: attempts and architecture

Section 3 supplied a measured plant model. This chapter develops the controller for it.

Multiple architectures were attempted from raw torque command to a structured gait generator while trying a variety of action spaces. The chapter closes with the final architecture, which is a Fourier gait generator.

4.1 Probleme formulation and general RL architecture

Formulation. The task is a single dash: run as fast as possible. Multiple formulation where attempted, from a simple dash with no care to stop it to a policy that could be steered with a joystick with the later proven much harder to train.

The learning algorithm was held fixed at PPO [21] from Stable-Baselines3, with an actor-critic MLP on CPU-parallel MuJoCo environments. Episodes are capped at 60 s. The control rate is 200 Hz, which Section 3.4.4 establishes as both the sensing ceiling and the deployable rate. A milestone is roughly 600 M environment steps and a multi-day cluster job, and the project as a whole spent roughly 3 G steps. That budget is large rather than asymptotic, and the distinction becomes load-bearing in Section 4.4.1.

The toy problem Committing a 12-DOF robot to an unproven pipeline is expensive. The complete chain was therefore exercised once at minimum stakes, on a single-motor inverted pendulum. That chain is simulate, train, export, deploy on embedded Linux, and close a loop over CAN. The observation is deliberately the hardware telemetry and nothing else: position in turns, velocity in turns per second, and torque, exactly as the drive reports them. The export bakes the observation normalisation statistics *into* the ONNX graph. The policy then runs on a Raspberry Pi under `onnxruntime`, closing the loop over CAN at 50 Hz with a watchdog.

Reward design.

4.2 The action-space ladder

Here are the different action spaces that were attempted, from the simplest to the most complex.

4.2.1 Direct joint torque

The first framework commanded joint torque directly, on a simplified biped, because the robot was still being designed. It was abandoned for two reasons, and the two failures are of different kinds.

It did not converge. The training history from that period is an unbroken sequence of reward-shaping attempts. The fall penalty rose because the robot learned to belly-flop. The alive bonus changed because it learned to dawdle. The feet-air-time thresholds were retuned because it learned to vibrate in place. The underlying problem is that a raw torque action space forces the policy to synthesise two things from scratch at every control step. One is the gravity compensation that keeps the robot standing. The other is the periodic structure that makes it walk. The reward landscape between those two is dominated by ways of falling over that convince the reward the robot is doing well.

It was undeployable Commanding torque means bypassing the drive’s internal loop, and streaming current setpoints over CAN. The robot was tested with a simple PID loop over the

CAN bus out of curiosity and as expected it's way too slow to command the actuator in torque over CAN. The leg was able to track points but it became jittery as soon as the gait to be tracked became faster.

4.2.2 Joint position targets

The second design made the action six position targets, one per actuated joint. The knee and ankle follow passively. In simulation these drive MuJoCo `<position>` actuators, with gains and force ranges from the datasheets. On hardware the action maps one to one onto what the drives accept.

4.2.3 Impedance with static gains

Once the impedance mode of the CubeMars drives was discovered, the action space was extended to include stiffness and damping.

4.2.4 A Fourier gait generator tuned by the policy in position space

4.2.5 A Fourier gait generator tuned by the policy in position and compliance space

4.2.6 Reflexes

4.2.7 Explicit oscillator against the Fourier series

The Fourier parameterisation is one member of a family. The other canonical member is the explicit limit-cycle oscillator of the Ijspeert school [8, 3]. The literature gave no basis to prefer one, so the question was measured. Sixteen cluster jobs shared one plant, one reward and one schedule, with only the generator swapped. Figure 4.1(a) shows the cold start on the pitch-free plant.

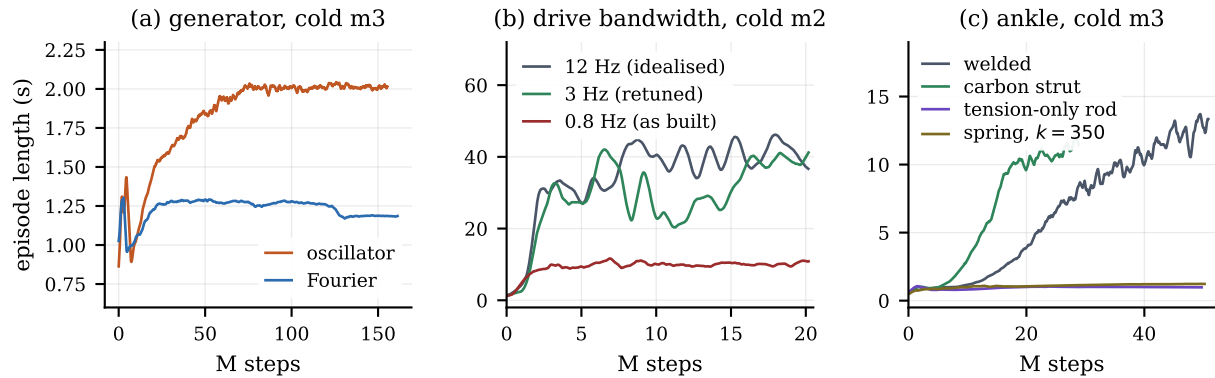


Figure 4.1: Learning curves of three controlled comparisons, episode length in seconds against environment steps: (a) generator family from a cold start on m3, oscillator against Fourier; (b) drive bandwidth from a cold start on m2, as built (0.8 Hz), retuned (3 Hz) and idealised (12 Hz); (c) ankle variants from a cold start on m3.

Building the oscillator arm forced a foot-space formulation with a measured inverse-kinematics lookup table. Constructing that table surfaced four plant facts. From the stance pose, *both* cam and thigh move the toe mostly fore-aft, and either sign of cam *shortens* the near-straight leg. The intuitive “thigh swings, cam lifts” mapping is therefore wrong, and it produces no step. The 4-bar has a folded solution branch that must be filtered out. The inversion must propagate

continuity between cells, or it flips branches mid-swing. And reachable foot lift at mid-stride is only about 8 cm, because the leg is near-straight there.

The verdict is a tie, with complementary strengths. The oscillator is more sample-efficient from a cold start on the hardest milestone. It reaches an episode length of 408 control steps against the Fourier arm’s 237, at 133 M steps. The Fourier generator is more reliable across seeds. On the warm-started pitch-free plant it crosses the line on 5 of 5 episodes at 2.45 m/s, against the oscillator’s 3 of 5 at 2.40 m/s.

The single best controller measured in the whole experiment was an oscillator run. It was the only arm of the sixteen that ever executed the stop phase, at 2.9 m/s with a left/right asymmetry of 0.08. Run up the full ladder, the oscillator cleared m1 to m4, with a 100 m dash at m2 and 2.95 m/s at m3. It then walled hard at m5, where roll is freed: episode length 337 against the Fourier ladder’s 3213 at matched budget, with the plateau confirmed at $2.5\times$ budget. Both share the identical three-parameter roll reflex, so the difference lies in the generator, and it points future work at lateral authority. The Fourier generator was kept for the deployment lineage. The oscillator result was used as tiebreak evidence that the action-space *family* is right.

One ablation refutes a hypothesis stated earlier in this chapter. Removing the per-step residual channel, leaving a pure oscillator, produced **stability without locomotion**. It survived 9700 of 12000 steps, but travelled 15 m of 72 at 0.25 m/s, with the worst cadence of all sixteen arms. The residual channel was the suspected fast open path behind a persistent 6 Hz footfall chatter. Removing it made the chatter worse, so that explanation is wrong. It is also the empirical form of the literature’s verdict that a per-step channel is mandatory. No equivalent arm was run on the Fourier generator, so the result constrains the family rather than this exact controller.

4.3 Curriculum design

Against the state of the art.

4.4 The pitch wall

4.4.1 A scripted baseline

What the comparison says about stability margin

4.4.2 Summary of negative results

What was tried	Outcome	What it rules out
Direct joint-torque learning (Section 4.2.1)	never converged; abandoned on a bandwidth argument later measured wrong	the action space was not the problem, the rejection premise was, and only instrumentation could tell
Per-step position policy for walking (Section 4.2.2)	skated at 32 M steps behind a healthy reward curve	reward curves cannot certify a gait; instrumented gait probes can
Emulated actuated ankle	strong version destabilised the gait; gentle version gave a small non-solving lift	a phase-blind torque fights the gait it stabilises, but the small positive identified the right lever
Decaying external pitch assist, four variants	crutch dependence every time; the unaided policy was worse than with no assist	any aid that holds the robot up will be leaned on, and the in-training curve hides it
Lowering the simulated centre of mass by 6–12 cm	slightly worse	rules out the cheapest candidate plant fix
Hard velocity, acceleration and cadence limits	collapsed the warm-started policy	they were fighting a broken internal default, not the gait
Centre-of-mass identification from excitation runs (Section 3.4.2)	rejected: explains $\leq 11\text{--}19\%$ of the gravity residual, 10 of 18 directions in the null space	the CAD centres of mass stand, and the experiment cannot resolve them
Per-body mass, load-dependent stiction, base tilt, pivot friction	all rejected as explanations of the same residual	the residual is conservative and gravity-correlated, so no friction model produces it
Pure oscillator without a residual channel (Section 4.2.7)	stability without locomotion: 15 m of 72, worst cadence in the set	refutes the residual channel as the cause of the 6 Hz footfall

Table 4.1: Approaches that produced no usable artefact, and what each one closes off. Each cost real time. In most cases the reason for the failure is more informative than a success would have been.

5 Results

This section is a recap of what this thesis achieved along with the result to characterize the deliverable.

5.1 Hardware delivered

Firstly, this thesis was quite involved in term of hardware and although the mechanical design was already done and provided, a lot of work was required to bring it from a fancy paper weight to a robot that could be controlled. The platform is a biped weighed at 15 kg. It carries four AKE90-8 QDD actuators (Table 3.1) and two AK60-39 for the hip roll. Its 13S2P pack holds 468 Wh and is rated 100 A continuous and 360 A for 2 s (Section 3.2.2). A Raspberry Pi 3B run the control loop at 200 Hz on a stock Linux. The platform can be backdriven and playback a recorded gait along with running the inference for a given policy. The platform was characterized to bound the performance envelope and to inform the training.

5.2 Running speed ceiling estimation

The physical speed ceiling was established independently of reinforcement learning and bounded to 6 m/s (Section 3.6). The bound was established by first principles and validated in simulation. The bound is a function of the leg workspace, the impulse it can deliver and the mass distribution of the robot.

5.3 Reinforcement learning results

The infrastructure to train a policy on the measured plant was built and tested. Multiple policy where trained with varying result that described in this chapter. The main result is that a the pipeline can converge to a running policy on the measured plant and reach speed of 3 m/s in simulation.

6 Discussion

This chapter measures the distance between the objective of Section 1.1 and the results of Section 5. It then ranks the changes that would close the gap.

6.1 Method-level lessons

Here are the lessons that were learned about the method proposed here.

Optimisers exploit objectives The policy gamed the reward by skating, penetrating the floor or vibrating on the floor. The scripted baseline’s coordinate-descent tuner gamed its own objective identically. Thus, objective design should be done with care and tested to find early signs that the objective is being gamed by the optimiser.

6.2 Future work

Now that the tooling is in place, the next project should start with a design search rather than a controller as it was originally intended. This project taught us the correct difficulty and time requirement prior. We now have hint about mechanical lever that should make the robot much easier to control and that should allow it to run faster. The first step would be to work on the workspace and its force map to find a better linkage and ankle stiffness. The second step would be to implement a foot that is more than a point contact to allow the robot to have a better leverage on the ground and be able to apply a torque and not only a force. The third step would be to implement and evaluate an impact/force sensor on the foot to allow the robot to have a better feedback on the ground contact and not having to wait for the base to fall over to react. Another step would be to increase the bus voltage to allow the robot to have a higher no-load speed and therefore be able to run faster if the torque is not saturated at that point.

7 Conclusion and future work

7.1 What was set out to do, and what was delivered

This thesis set out to apply reinforcement learning to a custom high-speed biped. The platform was inherited as a partially finished mechanical design. Three things were delivered.

A **working method**. It comprises the modelling, training and curriculum stack that produces a runner on the simulated plant.

A **working robot**. With calibration procedures, safety layer and a user interface.

A **measured robot**. To inform the training and to bound the performance envelope along with a reference for future work on this platform.

7.2 Limitations

7.3 Future work

Carbon blade foot that can start upright foot force / impact sensor Leg workspace redesign while accounting for the force map Rewrite of the training pipeline on GPU. replacement of the 3D printed parts with machined one.

8 Appendix

8.1 From CAD to a simulable model

The procedure below is the one that worked, after the detours recorded in Section 3.3. The repository holds the scripts and the exported files.¹

1. **Re-assemble in one CAD system.** Each limb is exported as STEP with its origin centred on its joint, then assembled and exported to URDF with a plugin. Inertia tensors supplied from a second CAD system are validated against their reference frame explicitly rather than trusted, because that export arrived rotated.
2. **Accept that the URDF is an approximation.** URDF is a tree format, so the pushrod floats and the 4-bar is open. The exported URDF is kept for tooling that needs it, and it is never simulated.
3. **Close the loop in MJCF.** Two sites, one at the pushrod tip and one on the shin, are tied by `<equality><connect>`. The residual at the measured workspace poses is 10^{-14} .
4. **Replace CAD masses with weighed masses,** and rescale each body’s inertia tensor by its own mass ratio. This removes the segment masses from the identification problem entirely.
5. **Re-settle the stance keyframe under gravity** after any plant change. A keyframe is an equilibrium of one particular ankle law, and re-settling is not idempotent: re-settling from an already crouched keyframe lands on the 4-bar’s fold branch.
6. **Gate every change with self-tests.** Analytic Jacobians against central differences, loop closure against the model’s own keyframe, and an assertion that a plant matches what its preset claims it is. A smoke suite builds and briefly runs all 123 training presets before any long launch.

8.2 Component databases

Both databases are published as standalone tools.

Actuator review.² It holds 140 commercial integrated actuators, 121 motors, 121 gearboxes and 9 standalone drives, as four joined tables. A candidate can therefore be composed rather than only looked up. Modelling follows Lee et al., *How to Model Brushless Electric Motors for the Design of Lightweight Robotic Systems* (arXiv:2310.00080).

Battery review.³ It holds roughly 430 cell models, 54 BMS modules and 110 commercial packs, each row carrying a source URL. A synthesiser builds a pack from cell, BMS and series-parallel count. “What I could build” therefore plots on the same axes as “what I can buy”. Every current rating carries a duration stamp, and falls into a continuous, pulse-at- t or fault tier. The pulse tier interpolates as $I(t) = \sqrt{I_{\text{cont}}^2 + (I_{\text{pulse}}^2 - I_{\text{cont}}^2) t_{\text{pulse}}/t}$. Fields inferred rather than read off a datasheet are written as ESTIMATED, so that they still parse and can be filtered out.

¹https://github.com/nathmo/running_robot

²<https://github.com/nathmo/actuatorReview>

³<https://github.com/nathmo/batteryReview>

8.3 Logbook

Timeline reconstructed from the repository’s commit history, with the exception of the measurement campaign (Section 3.4) which was performed in parallel to the training. The logbook is a record of the work, not a tutorial. It is therefore selective and omits the many dead ends, the many failed experiments, and the many commits that were only a few lines of hyper-parameter tuning.

- 2026-04-13** Repository created. First framework: MuJoCo plus Stable-Baselines3 PPO on a generic legged environment.
- 2026-04-20** A dozen reward and hyper-parameter commits in one day. The robot learns to belly-flop, and the fall penalty, forward-speed weight, alive bonus and feet-air-time thresholds are each chased in turn.
- 2026-04-21** Tuning starts going backwards. Decision to attack sim-to-real separately on a toy problem.
- 2026-04-30** Pendulum environment, PPO, ONNX export and a Raspberry Pi runtime: the project’s first complete simulation-to-hardware transfer (Section 4.1).
- 2026-05-21** The generic-robot approach is abandoned. The DASH-01 URDF is exported from CAD and the simulator rewritten around it.
- 2026-05-22** Manual URDF repair: the closed loop of the parallel leg cannot be expressed in URDF, and the export angles are wrong.
- 2026-06-22** First training on the real geometry: a standing policy, converged two days later.
- 2026-07-06** **Anti-skating rebuild.** An instrumented rollout measures what the policy is doing: both feet grounded 57–66 % of steps, median swing one control step, -0.09 m/s of travel under a $+0.5$ m/s command. Three causes fixed at once: contact `condim 3→6`, the six motor masses welded onto their stator bodies ($5.73→12.83$ kg), and a standing keyframe settled under gravity.
- 2026-07-06** Hardware bring-up begins in parallel: motor test scripts, slow sine sweeps, torque-control safety limits.
- 2026-07-09** Workspace and joint-limit calibration on the real leg.
- 2026-07-10** Web interface to drive the robot: per-motor sliders, live leg visualisation, gait recording and playback.
- 2026-07-14** Naming settled to DASH-01. Two structural changes: the base-DOF locking curriculum and the Fourier gait controller.
- 2026-07-21** m3 falls in 1.5 s. Pitch reflex, angular-momentum reward and normalisation rejuvenation added.
- 2026-07-22** The real m3 failure is an entropy-gate deadlock: the action standard deviation sat pinned at maximum for a whole 150 M-step run.
- 2026-07-23** The decaying pitch assist turns into a crutch. The control rate moves from 50 to 200 Hz, with the reward made rate-invariant and every curriculum count rescaled.
- 2026-07-24** A diagnostic proves the m3 policy plants its feet 8 cm behind the centre of mass while falling forward: it never learned the capture step. An evaluator bug that evaluated at 100 m rather than the curriculum distance is found and fixed.
- 2026-07-28** **Gait-frequency bug.** The 200 Hz rescale had widened the learnable range to (0.5, 50) Hz, so a neutral action meant a 25 Hz gait clock. Remapped to (0.5, 4.0).

- 2026-07-29** The fix produces the best m3 controller so far. The residual 6 Hz footfall is traced, wrongly at first, to ankle-spring resonance.
- 2026-07-30** Generator A/B result: no clear winner. The full ladder is relaunched twice, once with the cadence fixes and once on the oscillator generator.
- 2026-08-04** **The measurement campaign opens.** Weighed masses replace the CAD placeholders, 12.83→15.14 kg. The toe's static force capability is mapped over the workspace: 96 % of reach, 19:1 anisotropy, the passive ankle 8× short.
- 2026-08-05** System identification debugged: two sign errors, per-joint refitting, reduced Jacobian damping. Peak joint torque corrected to what the gearbox delivers.
- 2026-08-06** **The ankle question closes by statics.** The real spring deflects 26.6° and drives the heel 11.9 mm through the floor at half body weight. The CAN feedback protocol is measured, and a warm-start bug is found: seed replicates had been silent duplicates.
- 2026-08-09** Commanded speeds are bounded by what the plant can reach. The walker gets the base-DOF ladder it never had.
- 2026-08-10** **The plant becomes the robot as rebuilt.** Per-joint motor velocity caps are enforced for the first time. The ankle becomes a rigid carbon tube and the drive bandwidth is bracketed. On the hardware side, a failure that cost a joint prompts the three-tier flight recorder and the pre-move guard.
- 2026-08-11** First full ladder on the rebuilt plant: twelve jobs, about 23 h wall clock. m2 walks out the full 60 s cap; m5 is a hard wall. A scripted, no-learning walk controller is written as a baseline.
- 2026-08-19** **Foot-shape study: sixteen runs, negative result.** Reaching the answer took three wrong readings of the same data, and the harness's own rule, that a per-arm difference smaller than the within-rung spread is not a result, killed all of them.
- 2026-08-25** **Foot arc geometry measured.** Forward reach 31 cm, backward reach 2.7 cm. Re-solving the balanced keyframe takes the scripted controller at m3 from 0.91 s to 24.7 s.

References

- [1] Max A. Aebi. Dash-01: A High Speed Bipedal Running Robot. From Kinematic Design to Mechanical Implementation. Master’s thesis, EPFL, Lausanne, Switzerland, August 2026. Advisers: Mohamed Bouri, Alain Preneloup.
- [2] Associated Press. Chinese humanoid robot breaks speed record at world games. AP News, 2026. <https://apnews.com/article/chinese-humanoid-robot-beijing-world-games-record-069b76ee6748154fd67bec39fa927cf6>.
- [3] Guillaume Bellegarda and Auke Ijspeert. CPG-RL: Learning central pattern generators for quadruped locomotion. *IEEE Robotics and Automation Letters*, 2022. arXiv:2211.00458.
- [4] Miroslav Bogdanovic, Majid Khadiv, and Ludovic Righetti. Learning variable impedance control for contact sensitive tasks. *IEEE Robotics and Automation Letters*, 5(4):6129–6136, 2020. arXiv:1907.07500.
- [5] Devin Crowley, Jeremy Dao, Helei Duan, Kevin Green, Jonathan Hurst, and Alan Fern. Optimizing bipedal locomotion for the 100m dash with comparison to human running. *arXiv preprint*, 2025. arXiv:2508.03070.
- [6] Nicolas Heess, Dhruva TB, Srinivasan Sriram, Jay Lemmon, Josh Merel, Greg Wayne, Yuval Tassa, Tom Erez, Ziyu Wang, S. M. Ali Eslami, Martin Riedmiller, and David Silver. Emergence of locomotion behaviours in rich environments. *arXiv preprint*, 2017. arXiv:1707.02286.
- [7] Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26):eaau5872, 2019.
- [8] Auke Jan Ijspeert. Central pattern generators for locomotion control in animals and robots: a review. *Neural Networks*, 21(4):642–653, 2008.
- [9] Atil Iscen, Ken Caluwaerts, Jie Tan, Tingnan Zhang, Erwin Coumans, Vikas Sindhwani, and Vincent Vanhoucke. Policies modulating trajectory generators. *Conference on Robot Learning (CoRL)*, 2018. arXiv:1910.02812.
- [10] Benjamin Katz, Jared Di Carlo, and Sangbae Kim. Mini Cheetah: A platform for pushing the limits of dynamic quadruped control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6295–6301, 2019.
- [11] Scott Kuindersma, Robin Deits, Maurice Fallon, Andrés Valenzuela, Hongkai Dai, Frank Permenter, Twan Koolen, Pat Marion, and Russ Tedrake. Optimization-based locomotion planning, estimation, and control design for the Atlas humanoid robot. *Autonomous Robots*, 40(3):429–455, 2016.
- [12] Nathann Morand. actuatorReview. GitHub repository, 2026. Accessed 2026-08-30.
- [13] Nathann Morand. Motormap: An Interactive Actuator Explorer. Streamlit web application, 2026. Deployed instance of the actuatorReview repository, accessed 2026-08-30.
- [14] Nathann Morand. running_robot. GitHub repository, 2026. Workspace calibration, web UI and dynamic-identification pipeline; accessed 2026-09-01.
- [15] OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving Rubik’s Cube with a robot hand. *arXiv preprint*, 2019. arXiv:1910.07113.
- [16] Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. DeepMimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions on Graphics (SIGGRAPH)*, 37(4), 2018.
- [17] Nuofan Qiu, Chaoyang Song, and Fang Wan. Describing robots from design to learning: Towards an interactive lifecycle representation of robots. In *2024 International Conference on Advanced Robotics and Mechatronics (ICARM)*, pages 1081–1086, 2024.

- [18] Marc Raibert, Kevin Blankespoor, Gabriel Nelson, and Rob Playter. BigDog, the rough-terrain quadruped robot. In *IFAC Proceedings Volumes (17th IFAC World Congress)*, volume 41, pages 10822–10825, 2008.
- [19] Marc H. Raibert. *Legged Robots That Balance*. MIT Press, Cambridge, MA, 1986.
- [20] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Conference on Robot Learning (CoRL)*, pages 91–100, 2021.
- [21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint*, 2017. arXiv:1707.06347.
- [22] Nima Shafii, Siavash Aslani, Omid Mohamad Nezami, and Saeed Shiry. Evolution of biped walking using truncated Fourier series and particle swarm optimization. In *RoboCup 2009: Robot Soccer World Cup XIII*, volume 5949 of *Lecture Notes in Computer Science*, pages 344–354. Springer, 2010.
- [23] Jonah Siekmann, Yesh Godse, Alan Fern, and Jonathan Hurst. Sim-to-real learning of all common bipedal gaits via periodic reward composition. *IEEE International Conference on Robotics and Automation (ICRA)*, 2021. arXiv:2011.01387.
- [24] Jie Tan, Tingnan Zhang, Erwin Coumans, Atil Iscen, Yunfei Bai, Danijar Hafner, Steven Bohez, and Vincent Vanhoucke. Sim-to-real: Learning agile locomotion for quadruped robots. In *Robotics: Science and Systems (RSS)*, 2018. arXiv:1804.10332.
- [25] Waseda University. Robots of Waseda: WABOT-1. YouTube, May 2026. YouTube video, published May 7, 2026.
- [26] WhatIMakeOfThis. Asimo Walking. YouTube, July 2020. YouTube video, published July 16, 2020.
- [27] Zhaoming Xie, Patrick Clary, Jeremy Dao, Pedro Morais, Jonathan Hurst, and Michiel van de Panne. Learning locomotion skills for Cassie: Iterative design and sim-to-real. In *Conference on Robot Learning (CoRL)*, pages 317–329, 2019.